



DAY 14: CLOSURES IN JAVASCRIPT – A MUST-KNOW CONCEPT!



Lakshmiprasanna Vakada

Today, we will explore:

- ✓ What is a Closure?
- ✓ How Closures Work?
- ✓ Real-World Use Cases
- ✓ Common Interview Questions

I. What is a Closure?

A closure is a function that remembers the variables from its parent scope even after the parent function has finished executing.

💡 Think of it as a function "closing over" its environment and keeping access to the variables inside it.

✅ Example of Closure:

```
function outerFunction() {  
  let count = 0; // This variable is inside  
the outer function  
  
  return function innerFunction() {  
    count++; // innerFunction remembers  
`count`  
  }  
}
```

```
console.log(count);  
};  
}
```

```
const increment = outerFunction(); //  
`increment` now holds innerFunction  
increment(); // Output: 1  
increment(); // Output: 2  
increment(); // Output: 3
```

- ◆ Even though `outerFunction()` has finished executing, the `innerFunction()` still remembers the `count` variable!

2. How Closures Work?

When a function is returned from another function, it remembers the variables from its outer scope due to the Lexical Environment.

✓ Understanding Scope Retention

```
function greet(name) {  
  return function () {  
    console.log("Hello, " + name);  
  };  
}  
  
const sayHello = greet("Alice");  
sayHello(); // Output: "Hello, Alice"
```

- ◆ Even though `greet("Alice")` has finished execution, the function still remembers `name = "Alice"`.

3. Real-World Use Cases of Closures

✓ 1. Data Hiding & Encapsulation

Closures allow us to create private variables that cannot be accessed directly.

```
function createCounter() {  
  let count = 0;  
  
  return {  
    increment: function () {  
      count++;  
      console.log(count);  
    },  
    decrement: function () {  
      count--;  
      console.log(count);  
    },  
  },  
}
```

```
        getCount: function () {  
            return count;  
        }  
    };  
}  
  
const counter = createCounter();  
counter.increment(); // Output: 1  
counter.increment(); // Output: 2  
counter.decrement(); // Output: 1  
console.log(counter.getCount());  
// Output: 1
```

- ◆ Here, count is private and can only be modified through the returned methods.

✓ 2. Function Factories

Closures allow us to create pre-configured functions.

```
function power(exponent) {  
  return function (base) {  
    return Math.pow(base, exponent);  
  };  
}
```

```
const square = power(2);  
const cube = power(3);
```

```
console.log(square(4)); // Output: 16  
console.log(cube(2)); // Output: 8
```

- ◆ square is pre-configured to power of 2, while cube is pre-configured to power of 3.

✓ 3. Maintaining State in Event Listeners

Closures help retain state in event handlers.

```
function attachEventListener() {  
  let count = 0;  
  document.getElementById("btn").  
  addEventListener("click", function ()  
  {  
    count++;  
    console.log(`Button clicked  
    ${count} times`);  
  });  
}
```

`attachEventListener();`

- ◆ The count variable persists across multiple button clicks without polluting the global scope.

4. Common Interview Questions on Closures

✓ Question 1: What will be the output?

```
function outer() {  
  let count = 0;  
  return function inner() {  
    count++;  
    console.log(count);  
  };  
}  
  
const firstCall = outer();  
firstCall(); // ?  
firstCall(); // ?
```

Output:

1

2

✓ Question 2: What if we create another instance?

```
const secondCall = outer();  
secondCall(); // ?  
secondCall(); // ?
```

Output:

1
2

- ◆ Each call to `outer()` creates a new closure with its own count.